

Homework2

20251103415

September 27, 2025

1 Reward Setting and Discount Rate

The agent receives feedback from the environment in the form of rewards. The reward structure is designed to guide the agent towards the target state.

- **Target Reward:** A positive reward of $R_{\text{target}} = +1.0$ is given upon reaching the target state.
- **Forbidden Reward:** A negative reward of $R_{\text{forbidden}} = -1.0$ is given for attempting to enter a forbidden state or moving off the grid. The agent does not move in this case.
- **Step Reward:** A neutral reward of $R_{\text{step}} = 0.0$ is given for any other valid move.

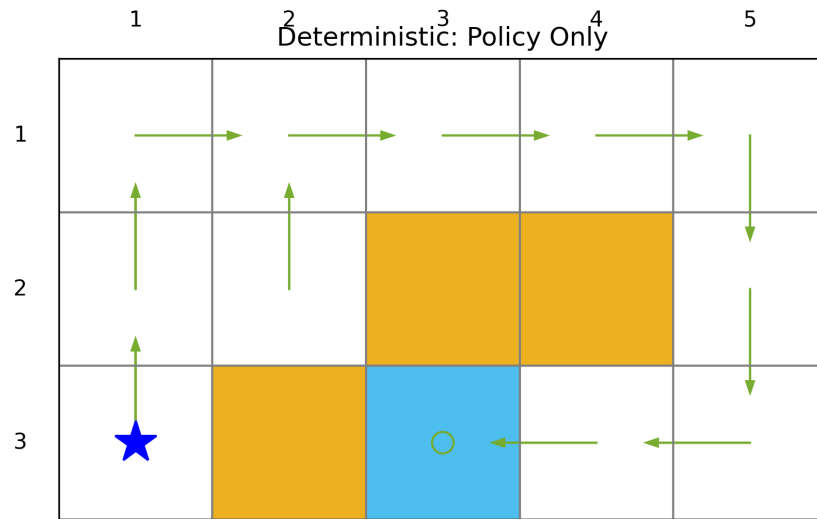
The discount rate, γ , is set to 0.9. This value determines the present value of future rewards, encouraging the agent to reach the target sooner rather than later. The discounted return is calculated as $G_t = \sum_{k=0}^{T-1} \gamma^k R_{t+k+1}$.

The environment is a custom-built Grid World, which is a discrete 5x3 grid (5 columns, 3 rows). The agent's objective is to navigate from a starting state to a target state while avoiding forbidden zones. The specific configuration is as follows:

- **Grid Size:** (Width, Height) = (5, 3)
- **Starting State:** (0, 2)
- **Target State:** (2, 2)
- **Forbidden States:** [(2, 1), (1, 2), (3, 1)]

2 Policy 1: Deterministic Policy

2.1 Policy Plot



2.2 vector r_π and P_π

Reward vector r_π

$$r_\pi = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0]$$

Table 1: and transition matrix P_π for the deterministic policy.

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	1	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0	0	0
4	0	0	1	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	1	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0	0	0
7	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	0
9	0	0	0	0	0	0	0	0	0	1	0	0
10	0	0	0	0	0	0	0	0	0	0	0	1
11	0	0	0	0	0	0	0	0	1	0	0	0

2.3 Closed-form algorithm

For a fixed policy π , the value vector satisfies $V_\pi = r_\pi + \gamma P_\pi V_\pi$, i.e., $(I - \gamma P_\pi)V_\pi = r_\pi$. With $0 \leq \gamma < 1$ and row-stochastic P_π , $I - \gamma P_\pi$ is invertible and

$$V_\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

In our grid, we construct r_π and P_π from the same transition rules as Assignment 1 (goal absorbing, forbidden states stationary) and solve the linear system via a direct solver. We then overlay V_π on the grid.

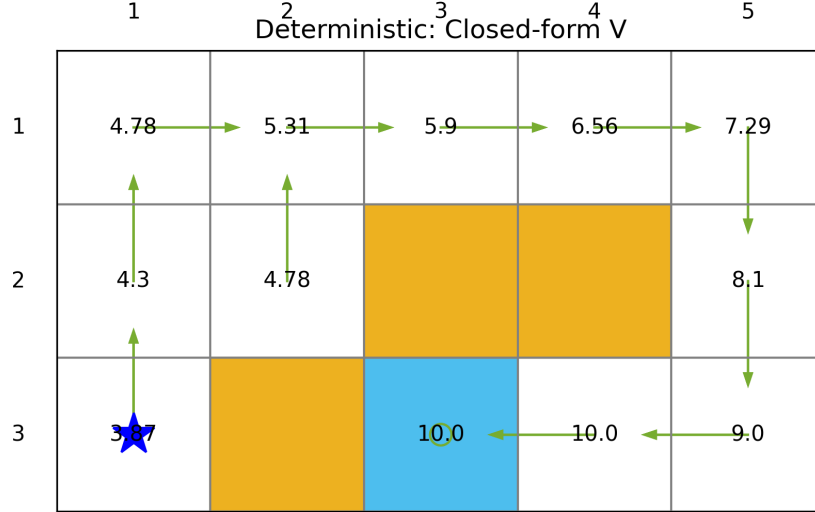


Figure 2: Visualization of the deterministic policy and state value start state (0, 2).

2.4 Iterative algorithm

Starting from $V^{(0)} = 0$, we iterate

$$V^{(k+1)} = r_\pi + \gamma P_\pi V^{(k)}$$

until $\|V^{(k+1)} - V^{(k)}\|_\infty < \varepsilon$. The operator $T(V) = r_\pi + \gamma P_\pi V$ is a γ -contraction, so $V^{(k)} \rightarrow V_\pi$. We plot the converged values on the grid. In our experiments, the iterative and closed-form maps coincide up to floating-point error, as expected.

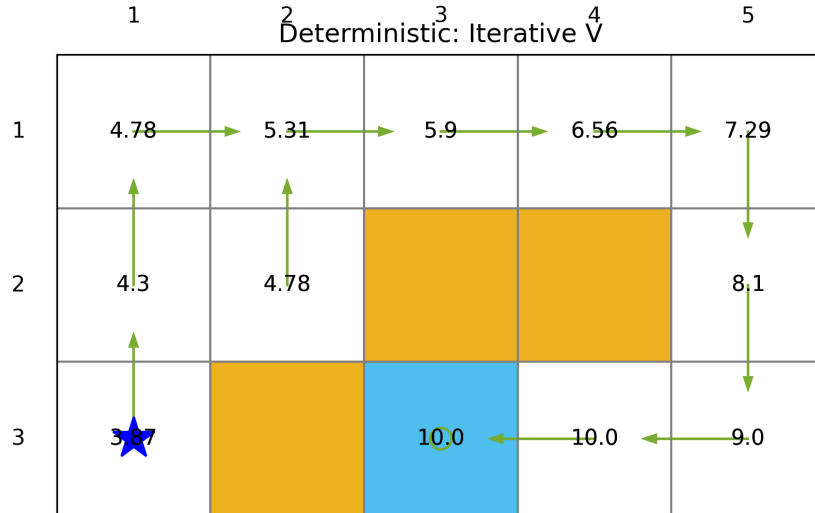


Figure 3: Visualization of the deterministic policy (arrows) start state (0, 2).

3 Policy 2: Stochastic Policy

3.1 Policy Plot

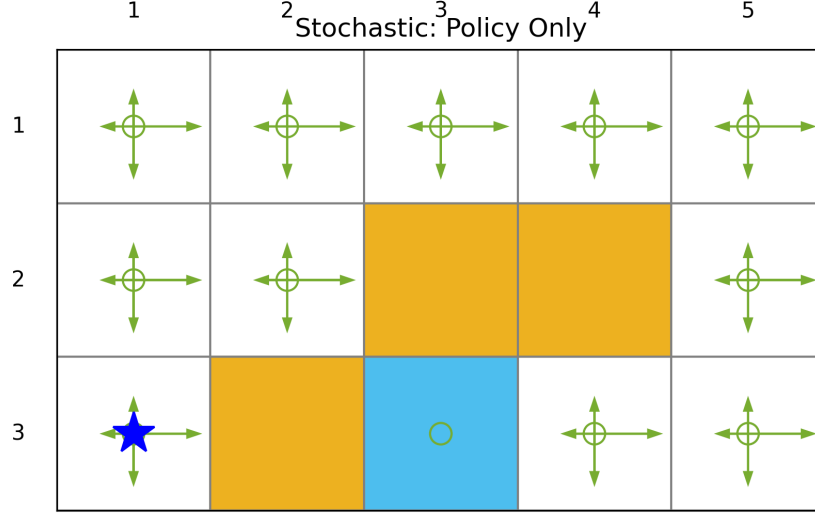


Figure 4: Visualization of the deterministic policy (arrows) start state (0, 2).

3.2 vector r_π and P_π

Reward vector r_π

$$r_\pi = \begin{bmatrix} -0.15 & -0.05 & -0.85 & -0.10 & -0.80 & -0.40 & 1.00 & -0.40 & -0.35 & -0.60 & -0.55 & -0.80 \end{bmatrix}$$

Table 2: and transition matrix P_π for the deterministic policy.

	0	1	2	3	4	5	6	7	8	9	10	11
0	0.20	0.30	0.00	0.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1	0.10	0.10	0.30	0.00	0.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00
2	0.00	0.10	0.90	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
3	0.05	0.00	0.00	0.15	0.30	0.50	0.00	0.00	0.00	0.00	0.00	0.00
4	0.00	0.05	0.00	0.10	0.85	0.00	0.00	0.00	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.05	0.00	0.45	0.00	0.50	0.00	0.00	0.00	0.00
6	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00
7	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.45	0.00	0.50	0.00	0.00
8	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.45	0.00	0.00	0.50
9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.65	0.30	0.00
10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.60	0.30
11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.10	0.85

3.3 Closed-form algorithm

For a fixed policy π , the value vector satisfies $V_\pi = r_\pi + \gamma P_\pi V_\pi$, i.e., $(I - \gamma P_\pi)V_\pi = r_\pi$. With $0 \leq \gamma < 1$ and row-stochastic P_π , $I - \gamma P_\pi$ is invertible and

$$V_\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

In our grid, we construct r_π and P_π from the same transition rules as Assignment 1 (goal absorbing, forbidden states stationary) and solve the linear system via a direct solver. We then overlay V_π on the grid.

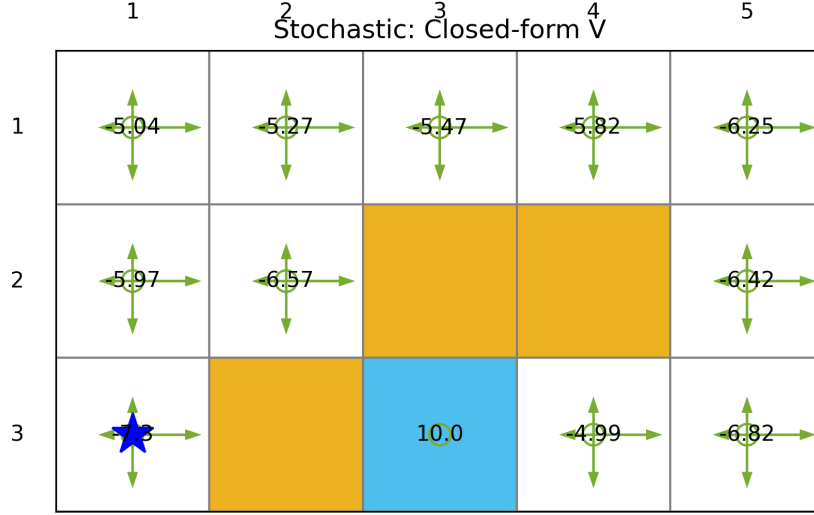


Figure 5: Visualization of the stochastic policy and state value start state $(0, 2)$.

3.4 Iterative algorithm

Starting from $V^{(0)} = 0$, we iterate

$$V^{(k+1)} = r_\pi + \gamma P_\pi V^{(k)}$$

until $\|V^{(k+1)} - V^{(k)}\|_\infty < \varepsilon$. The operator $T(V) = r_\pi + \gamma P_\pi V$ is a γ -contraction, so $V^{(k)} \rightarrow V_\pi$. We plot the converged values on the grid. In our experiments, the iterative and closed-form maps coincide up to floating-point error, as expected.

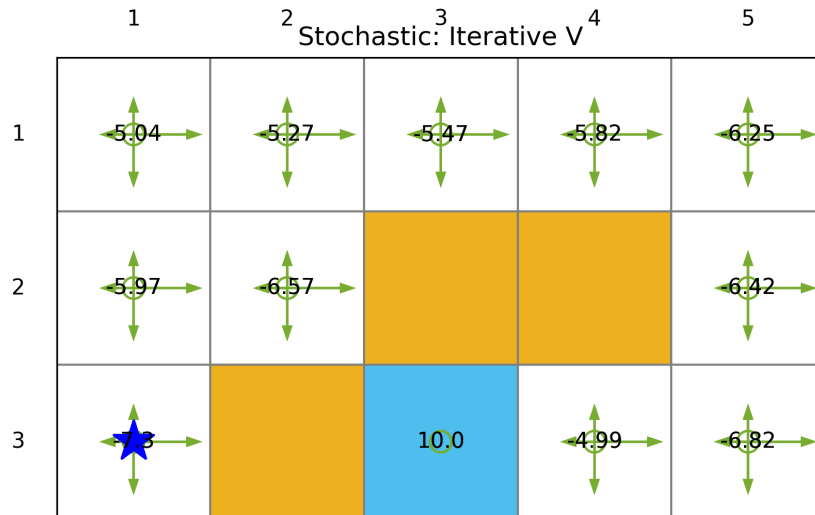


Figure 6: Visualization of the Stochastic policy (arrows) start state $(0, 2)$.

4 Key codes for Closed-form and Iterative algorithm

Closed-form evaluation. The closed-form method solves the linear system

$$(I - \gamma P_\pi)V = r_\pi$$

directly to obtain the exact value vector V_π up to numerical precision.

```
1 def policy_eval_closed_form(r_pi, P_pi, gamma):
2     I = np.eye(P_pi.shape[0])
3     V = np.linalg.solve(I - gamma * P_pi, r_pi)
4     return V
```

Listing 1: Closed-form policy evaluation

This function builds the identity matrix I and uses NumPy’s `solve` to compute $V_\pi = (I - \gamma P_\pi)^{-1} r_\pi$ efficiently, without explicitly inverting a matrix.

Iterative evaluation. The iterative method repeatedly applies the Bellman operator

$$V^{(k+1)} = r_\pi + \gamma P_\pi V^{(k)}$$

until convergence.

```
1 def policy_eval_iterative(r_pi, P_pi, gamma, tol=1e-8, max_iters=10000)
2     :
3     V = np.zeros_like(r_pi)
4     for k in range(max_iters):
5         V_new = r_pi + gamma * P_pi.dot(V)
6         if np.max(np.abs(V_new - V)) < tol:
7             return V_new, k+1
8     V = V_new
9     return V, max_iters
```

Listing 2: Iterative (Bellman) policy evaluation

Here we initialize V with zeros and update it until the maximum change is smaller than the tolerance `tol`. This guarantees convergence to the same V_π as the closed-form method, with iteration count depending on γ and the tolerance.

5 Discussion of Observations

Closed-form vs. Iterative algorithms. Both methods are designed to evaluate the same policy. The closed-form algorithm solves the linear system $(I - \gamma P_\pi)V_\pi = r_\pi$ directly, producing exact values up to numerical precision. The iterative algorithm repeatedly applies the Bellman operator $T(V) = r_\pi + \gamma P_\pi V$ until convergence. In practice, the two approaches yield numerically identical state values, with differences only at the scale of floating-point error. This confirms that both converge to the same unique fixed point V_π .

Deterministic vs. Stochastic policies. Under the deterministic policy, the agent always follows the shortest path to the goal. As a result, states along the optimal trajectory exhibit high values, and the value landscape forms a clear gradient pointing toward the goal. In contrast, the stochastic policy assigns probability mass to multiple actions, including suboptimal ones. This introduces a risk of wandering or entering forbidden states, lowering the expected return. Consequently, the value function is flatter, with reduced values even for states close to the goal.

Key insight. While both algorithms produce the same V_π , the choice of policy strongly affects the value distribution. Deterministic policies lead to sharper and higher state values along direct paths to the goal, whereas stochastic policies yield more diffuse and conservative value estimates due to uncertainty in action selection.